



CANLI FALCILAR — Flutter Entegrasyon Prompt'u

Uygulama: canlifal_social (Flutter)

Backend: canlifal.com

Base URL: https://canlifal.com

Auth: Tüm isteklerde Authorization: Bearer {JWT} header'ı zorunlu

Tarih: Haziran 2025



İÇİNDEKİLER

1. Genel Mimari & Akış
2. Veri Modelleri
3. Falcı Listeleme & Profil
4. Falcı Başvurusu
5. Falcı Kendi Profili & Online Durumu
6. Seans Oluşturma (Kullanıcı → Falcı)
7. Gelen Talep Yönetimi (Falcı Tarafı)
8. Aktif Seans Kontrolü (Kullanıcı Tarafı)
9. Canlı Oda — Room Bilgisi
10. Canlı Oda — Mesajlaşma
11. Canlı Oda — Timer & Ping
12. Canlı Oda — Süre Uzatma
13. Canlı Oda — Seans Bitirme
14. WebRTC Sinyalizasyon
15. Değerlendirme & Yorumlar
16. Hediyeler & Ödüller
17. Push Notification Entegrasyonu
18. Tam Akış Senaryoları
19. Hata Kodları & Edge Case'ler
20. Flutter Kod İskeletleri

1. Genel Mimari & Akış

1.1 Sistemdeki Roller

Rol	Açıklama
Kullanıcı (User)	Falcılardan seans talep eder, jeton öder
Falcı (Teller)	Gelen talepleri kabul/reddeder, canlı fal bakar
Admin/Yönetici (Staff)	Jeton düşülmez, ücretsiz kullanır

1.2 Ana Akış Diyagramı

1. Kullanıcı falcı listesini görür
2. Falcı seçer → Seans talebi gönderir (jeton düşülür)
3. Falcıya push notification gider
4. Falcı kabul/reddetme ekranı görür
- 5a. KABUL → roomId oluşur, status: active
- 5b. RED → jetonlar iade edilir, status: cancelled
6. Kabul sonrası her iki taraf canlı odaya girer
7. Falcı timer'ı başlatır
8. Mesajlaşma + WebRTC (sesli/görüntülü) başlar
9. Ping ile süre takibi (her dakika)
10. Süre dolmadan kullanıcı uzatabilir
11. Seans biter → kullanılmayan jetonlar iade edilir
12. Komisyon hesaplanır, falcı kazancı güncellenir
13. Kullanıcı değerlendirme bırakır

1.3 Jeton Sistemi

- Platform ayarı: `credits_per_minute` (varsayılan: **10 jeton/dakika**)
- Komisyon oranı: `commission_rate` (varsayılan: **%20**)
- Staff (admin/yönetici) kullanıcılar jeton ödemez
- Kullanılmayan süre seansın sonunda iade edilir

2. Veri Modelleri

2.1 LiveFortuneTeller (Falcı Profili)

```
class LiveFortuneTeller {
    final String id;
    final String userId;
    final String displayName;
    final String? bio;
    final List<String> specialties;
    final int pricePerSession;           // varsayılan: 100
    final double rating;                 // varsayılan: 5.0
    final int totalSessions;
    final int totalReviews;
    final bool isOnline;
    final bool isVerified;
    final bool isActive;
    final String? avatar;
    final String applicationStatus;     // pending, approved, rejected
    final bool isBanned;
    final bool isFrozen;
    final int totalEarnings;
    final int bonusCredits;
    final String tellerLevel;           // bronze, silver, gold, diamond
    final int levelPoints;

    // İzinler (admin tarafından yönetilir)
    final bool canGoOnline;
    final bool canChat;
    final bool canStartSession;
    final bool canSetPrice;
    final bool canEditProfile;
    final bool canViewEarnings;
    final bool canWithdraw;
    final int maxSessionsPerDay;        // varsayılan: 10
    final int commissionRate;          // varsayılan: 20

    // Kimlik doğrulama
    final String? verificationDocUrl;
    final String verificationStatus;    // none, pending, approved, rejected

    final DateTime createdAt;
    final DateTime? approvedAt;
}
```

2.2 LiveSession (Seans)

```
class LiveSession {
    final String id;
    final String tellerId;
    final String userId;
    final String fortuneType;           // coffee, tarot, astrology, palmistry,
    numerology, general
    final String status;               // pending, active, completed, cancelled
    final int creditsCharged;          // toplam düşülen jeton
    final int maxMinutes;              // toplam izin verilen dakika
    final int minutesUsed;             // kullanılan dakika
    final int creditsPerMinute;        // dakika başına jeton ücreti
    final String? roomId;              // "room_{sessionId}_{timestamp}"
    final bool timerStarted;
    final DateTime? timerStartedAt;
    final DateTime? startedAt;
    final DateTime? endedAt;
    final DateTime? lastPingAt;
    final DateTime createdAt;

    // ilişkiler
    final LiveFortuneTeller? teller;
    final UserBasic? user;
}
```

2.3 LiveSessionMessage (Mesaj)

```
class LiveSessionMessage {
    final String id;
    final String sessionId;
    final String senderId;
    final String message;
    final DateTime createdAt;
}
```

2.4 RoomSignal (WebRTC Sinyal)

```
class RoomSignal {
    final String id;
    final String sessionId;
    final String senderId;
    final String receiverId;
    final String signalType;           // "offer", "answer", "ice-candidate"
    final dynamic signalData;          // JSON parse edilmiş WebRTC verisi
    final bool processed;
    final DateTime createdAt;
}
```

2.5 TellerChatSession

```
class TellerChatSession {
    final String id;
    final String liveSessionId;
    final String userId;
    final String tellerId;
    final String status; // "active", "closed"
    final DateTime createdAt;
    final DateTime? closedAt;
}
```

2.6 LiveTellerReview (Değerlendirme)

```
class LiveTellerReview {
    final String id;
    final String tellerId;
    final String sessionId; // unique – her seans için 1 yorum
    final int rating; // 1-5
    final String? comment;
    final DateTime createdAt;
    // İlişki
    final LiveSession? session; // session.user bilgisi içerir
}
```

2.7 Fal Tipleri

```
enum FortuneType {
    coffee('coffee', 'Kahve Falı'),
    tarot('tarot', 'Tarot'),
    astrology('astrology', 'Astroloji'),
    palmistry('palmistry', 'El Falı'),
    numerology('numerology', 'Numeroloji'),
    general('general', 'Genel Danışmanlık');

    final String key;
    final String label;
    const FortuneType(this.key, this.label);
}
```

3. Falcı Listeleme & Profil

3.1 Falcı Listesi

GET /api/fortune-tellers

Query Parametreleri:

Parametre	Tip	Zorunlu	Açıklama
online	"true"	Hayır	Sadece çevrimiçi fal-cıları getir
specialty	string	Hayır	Uzmanlık alanına göre filtrele ("tarot", "coffee" vb.)
sort	string	Hayır	"rating", "sessions", "newest"

Örnek İstek:

```
GET /api/fortune-tellers?online=true&specialty=tarot&sort=rating
Authorization: Bearer {JWT}
```

Başarılı Yanıt (200):

```
[
  {
    "id": "clxxx1",
    "userId": "user_abc",
    "displayName": "Astroloji Uzmanı Ayşe",
    "bio": "15 yıllık deneyimle astroloji ve tarot...",
    "specialties": ["tarot", "astrology"],
    "pricePerSession": 100,
    "rating": 4.8,
    "totalSessions": 250,
    "totalReviews": 180,
    "isOnline": true,
    "isVerified": true,
    "isActive": true,
    "avatar": "https://canlifal.com/uploads/avatar.jpg",
    "tellerLevel": "gold",
    "levelPoints": 5200,
    "user": {
      "id": "user_abc",
      "name": "Ayşe Yılmaz",
      "image": "https://pbs.twimg.com/profile_images/1939451430223417344/lNhZ7UBI.jpg"
    },
    "queuePosition": null
  }
]
```

Not: Yanıt 60 saniye önbelleğe alınır (stale-while-revalidate). `online=true` ise önbellek atlanır.

3.2 Tekil Falcı Profili

```
GET /api/fortune-tellers/{tellerId}
```

Örnek İstek:

```
GET /api/fortune-tellers/clxxx1
Authorization: Bearer {JWT}
```

Başarılı Yanıt (200):

```
{
  "id": "clxxx1",
  "userId": "user_abc",
  "displayName": "Astroloji Uzmanı Ayşe",
  "bio": "15 yıllık deneyimle...",
  "specialties": ["tarot", "astrology"],
  "pricePerSession": 100,
  "rating": 4.8,
  "totalSessions": 250,
  "totalReviews": 180,
  "isOnline": true,
  "isVerified": true,
  "isActive": true,
  "avatar": "https://thumbs.dreamstime.com/b/finance-451760037.jpg",
  "tellerLevel": "gold",
  "levelPoints": 5200,
  "createdAt": "2025-01-15T...",
  "reviews": [
    {
      "id": "rev1",
      "rating": 5,
      "comment": "Harika bir deneyimdi!",
      "createdAt": "2025-06-10T...",
      "session": {
        "user": {
          "id": "user_xyz",
          "name": "Mehmet",
          "image": null,
          "username": "mehmet42"
        },
        "fortuneType": "tarot"
      }
    }
  ],
  "user": {
    "id": "user_abc",
    "name": "Ayşe Yılmaz",
    "image": "https://pbs.twimg.com/media/HITzfW1WAAk8-p.jpg"
  }
}
```

4. Falcı Başvurusu

```
POST /api/fortune-tellers/apply
```

İstek Body:

```
{
  "displayName": "Tarot Uzmanı Fatma",
  "bio": "10 yıllık tarot deneyimi...",
  "specialties": ["tarot", "coffee"],
  "applicationNote": "Sertifikalarım mevcut, detay için..."
}
```

Alan	Tip	Zorunlu	Açıklama
displayName	string	✓	Görünen ad
bio	string	Hayır	Biyografi
specialties	string[]	✓	En az 1 uzmanlık alanı
applicationNote	string	Hayır	Başvuru notu

Başarılı Yanıt (200):

```
{
  "success": true,
  "teller": {
    "id": "clyyy1",
    "userId": "user_123",
    "displayName": "Tarot Uzmanı Fatma",
    "applicationStatus": "pending",
    "isActive": false,
    "isVerified": false,
    "commissionRate": 20,
    "maxSessionsPerDay": 10,
    "canGoOnline": true,
    "canChat": true,
    "canStartSession": true,
    "canSetPrice": false,
    "canEditProfile": true,
    "canViewEarnings": true,
    "canWithdraw": false
  }
}
```

Hata Yanıtları:

- 400 : "You already have an application" — Zaten başvuru var
- 400 : "Display name and at least one specialty required" — Eksik alan

5. Falcı Kendi Profili & Online Durumu

5.1 Kendi Profili Görme

```
GET /api/fortune-tellers/my-profile
```

Başarılı Yanıt (200):

```
{
  "id": "clxxx1",
  "displayName": "Astroloji Uzmanı Ayşe",
  "bio": "...",
  "avatar": "https://placeholder.co/1200x600/e2e8f0/1e293b?text=portrait_photo_of_Astroloji_Uzman_Ay_e_a_female_",
  "specialties": ["tarot", "astrology"],
  "pricePerSession": 100,
  "rating": 4.8,
  "totalSessions": 250,
  "isOnline": true,
  "isVerified": true,
  "isActive": true,
  "applicationStatus": "approved",
  "totalEarnings": 50000,
  "bonusCredits": 500,
  "isBanned": false,
  "isFrozen": false,
  "canGoOnline": true,
  "canChat": true,
  "canStartSession": true,
  "canSetPrice": false,
  "canEditProfile": true,
  "canViewEarnings": true,
  "canWithdraw": false,
  "maxSessionsPerDay": 10,
  "commissionRate": 20,
  "createdAt": "2025-01-15T..."
}
```

Hata: 404 — Falcı profili bulunamadı (kullanıcı falcı değil)

5.2 Online/Offline Durumu Toggle

POST /api/fortune-tellers/toggle-online

İstek Body (opsiyonel):

```
{
  "isOnline": true
}
```

Body boş bırakılırsa mevcut durumun tersi yapılır (toggle).

Başarılı Yanıt (200):

```
{
  "isOnline": true,
  "message": "Now online"
}
```

Hata Yanıtları:

- 403 : "Application not approved" — Başvuru onaylanmamış

- 403 : "Account is banned" — Hesap yasaklı
- 404 : "Not a fortune teller" — Falcı profili yok

5.3 Online Durumu Sorgulama

GET /api/fortune-tellers/toggle-online

Başarılı Yanıt (200) — Falcı ise:

```
{
  "isTeller": true,
  "id": "clxxx1",
  "isOnline": true,
  "applicationStatus": "approved",
  "isBanned": false,
  "displayName": "Astroloji Uzmanı Ayşe"
}
```

Başarılı Yanıt (200) — Falcı değilse:

```
{
  "isTeller": false
}
```

6. Seans Oluşturma (Kullanıcı → Falcı)

6.1 Seans Talebi Gönderme

POST /api/fortune-tellers/session

⚠ Flutter için bu endpoint'i kullanın (/api/fortune-tellers/session). `tellerId` body içinde gönderilir.

İstek Body:

```
{
  "tellerId": "clxxx1",
  "fortuneType": "tarot",
  "duration": 10
}
```

Alan	Tip	Zorunlu	Varsayılan	Açıklama
tellerId	string	✓	—	Falçı ID'si
fortuneType	string	Hayır	"general"	Fal tipi
duration	int	Hayır	10	Dakika cinsinden süre

Arka Plan İşlemleri:

1. Kullanıcının jeton bakiyesi kontrol edilir
2. `duration × credits_per_minute` kadar jeton düşülür
3. `LiveSession` oluşturulur (status: "pending")
4. Falçıya push notification gönderilir

Başarılı Yanıt (201):

```
{
  "success": true,
  "sessionId": "sess_abc123",
  "session": {
    "id": "sess_abc123",
    "tellerId": "clxxx1",
    "userId": "user_456",
    "fortuneType": "tarot",
    "status": "pending",
    "creditsCharged": 100,
    "maxMinutes": 10,
    "creditsPerMinute": 10,
    "roomId": null,
    "timerStarted": false,
    "createdAt": "2025-06-17T12:00:00Z"
  }
}
```

Hata Yanıtları:

- 400 : "tellerId gerekli" — tellerId eksik
- 400 : "Falçı müsait değil" — Falçı aktif/doğrulanmış değil
- 400 : "Yetersiz jeton bakiyesi" — Jeton yetersiz
- 401 : "Oturum açmanız gerekiyor" — Auth hatası
- 404 : "Kullanıcı bulunamadı"

6.2 Seans Detayı Sorgulama

```
GET /api/fortune-tellers/session?sessionId={sessionId}
```

Başarılı Yanıt (200):

```
{
  "id": "sess_abc123",
  "tellerId": "clxxx1",
  "userId": "user_456",
  "fortuneType": "tarot",
  "status": "active",
  "creditsCharged": 100,
  "maxMinutes": 10,
  "minutesUsed": 3,
  "creditsPerMinute": 10,
  "roomId": "room_sess_abc123_1718618400000",
  "timerStarted": true,
  "timerStartedAt": "2025-06-17T12:01:00Z",
  "startedAt": "2025-06-17T12:00:30Z",
  "createdAt": "2025-06-17T12:00:00Z",
  "teller": {
    "id": "clxxx1",
    "userId": "user_abc",
    "displayName": "Astroloji Uzmanı Ayşe",
    "specialties": ["tarot", "astrology"],
    "avatar": "https://placeholder.co/1200x600/e2e8f0/1e293b?text=Profile_avatar_image_of_Astroloji_Uzman_Ay_e_an_"
  },
  "user": {
    "id": "user_456",
    "name": "Mehmet K.",
    "image": null
  }
}
```

6.3 Kullanıcının Tüm Seansları

GET /api/fortune-tellers/session

`sessionId` parametresi olmadan çağrılırsa, kullanıcının son 50 seansını döner.

Başarılı Yanıt (200):

```
[
  {
    "id": "sess_abc123",
    "fortuneType": "tarot",
    "status": "completed",
    "creditsCharged": 80,
    "maxMinutes": 10,
    "minutesUsed": 8,
    "createdAt": "2025-06-17T...",
    "teller": {
      "id": "clxxx1",
      "displayName": "Astroloji Uzmanı Ayşe",
      "avatar": "https://..."
    }
  }
]
```

7. Gelen Talep Yönetimi (Falcı Tarafı)

7.1 Bekleyen Seansları Polling

Falcı uygulamada çevrimiçi olduğu sürece bu endpoint'i **3-5 saniyede bir** poll etmelidir.

```
GET /api/fortune-tellers/sessions?status=pending
```

Başarılı Yanıt (200):

```
[
  {
    "id": "sess_abc123",
    "userId": "user_456",
    "fortuneType": "tarot",
    "status": "pending",
    "creditsCharged": 100,
    "maxMinutes": 10,
    "creditsPerMinute": 10,
    "createdAt": "2025-06-17T12:00:00Z",
    "user": {
      "id": "user_456",
      "name": "Mehmet K.",
      "image": "https://pbs.twimg.com/profile_images/1374748719988543491/5a5__EqT.jpg"
    }
  }
]
```

7.2 Talebi Kabul Etme

```
PATCH /api/fortune-tellers/sessions/{sessionId}
```

İstek Body:

```
{
  "action": "accept"
}
```

Arka Plan İşlemleri:

1. `roomId` oluşturulur: `"room_{sessionId}_{timestamp}"`
2. `TellerChatSession` oluşturulur (status: `"active"`)
3. `status` → `"active"`, `startedAt` → şimdi
4. `creditsPerMinute` platform ayarından atanır
5. Kullanıcıya push notification gönderilir

Başarılı Yanıt (200):

```
{
  "id": "sess_abc123",
  "status": "active",
  "roomId": "room_sess_abc123_1718618400000",
  "startedAt": "2025-06-17T12:00:30Z",
  "maxMinutes": 5,
  "creditsPerMinute": 10,
  "timerStarted": false
}
```

Hata: 400 : "Session is not pending" — Zaten kabul edilmiş/iptal edilmiş

7.3 Talebi Reddetme

PATCH /api/fortune-tellers/sessions/{sessionId}

İstek Body:

```
{
  "action": "reject"
}
```

Arka Plan İşlemleri:

1. status → "cancelled"
2. Kullanıcının jetonları **tam iade** edilir
3. Kullanıcıya "reddedildi" bildirimi gider

Başarılı Yanıt (200):

```
{
  "id": "sess_abc123",
  "status": "cancelled",
  "endedAt": "2025-06-17T12:01:00Z"
}
```

7.4 Talebi İptal Etme

PATCH /api/fortune-tellers/sessions/{sessionId}

İstek Body:

```
{
  "action": "cancel"
}
```

cancel ve reject aynı işlevi yapar — jetonlar iade edilir. Fark sadece notification mesajındadır.

8. Aktif Seans Kontrolü (Kullanıcı Tarafı)

Kullanıcı uygulamayı açtığında veya belirli aralıklarla (push notification aldığı anda) aktif seansları kontrol etmelidir.

```
GET /api/user/active-sessions
```

Başarılı Yanıt (200):

```
[
  {
    "id": "sess_abc123",
    "fortuneType": "tarot",
    "status": "active",
    "maxMinutes": 10,
    "minutesUsed": 0,
    "createdAt": "2025-06-17T12:00:00Z",
    "startedAt": "2025-06-17T12:00:30Z",
    "roomId": "room_sess_abc123_1718618400000",
    "teller": {
      "id": "clxxx1",
      "displayName": "Astroloji Uzmanı Ayşe",
      "avatar": "https://upload.wikimedia.org/wikipedia/commons/7/7a/East_side_of_stela_C%2C_Quirigua.PNG"
    }
  }
]
```

Aktif seans varsa kullanıcıyı otomatik olarak canlı oda ekranına yönlendirin.

9. Canlı Oda — Room Bilgisi

Seans kabul edildikten sonra her iki taraf bu endpoint'i çağırarak oda bilgisini alır.

```
GET /api/room/{sessionId}
```

Başarılı Yanıt (200):

```
{
  "id": "sess_abc123",
  "tellerId": "clxxx1",
  "userId": "user_456",
  "fortuneType": "tarot",
  "status": "active",
  "creditsCharged": 100,
  "maxMinutes": 10,
  "minutesUsed": 3,
  "creditsPerMinute": 10,
  "roomId": "room_sess_abc123_1718618400000",
  "timerStarted": true,
  "timerStartedAt": "2025-06-17T12:01:00Z",
  "startedAt": "2025-06-17T12:00:30Z",
  "lastPingAt": "2025-06-17T12:04:00Z",
  "elapsedSeconds": 180,
  "isUser": true,
  "isTeller": false,
  "peerId": "user_abc",
  "teller": {
    "id": "clxxx1",
    "displayName": "Astroloji Uzmanı Ayşe",
    "avatar": "https://aweinspired.com/cdn/shop/files/Oshun_Collector_Lariat_GV_ON_FIG.webp?v=1781291324&width=3840",
    "user": {
      "id": "user_abc",
      "name": "Ayşe Yılmaz",
      "image": "https://ars.els-cdn.com/content/image/1-s2.0-S089062382600016X-gr2.jpg"
    }
  },
  "user": {
    "id": "user_456",
    "name": "Mehmet K.",
    "image": null,
    "jetonBalance": 500,
    "membership": "premium"
  }
}
```

Önemli Alanlar:

- `isUser` / `isTeller` : Mevcut kullanıcının rolünü belirler
- `peerId` : WebRTC için karşı tarafın `userId`'si
- `elapsedSeconds` : Timer başladıysa sunucunun hesapladığı geçen saniye
- `timerStarted` + `timerStartedAt` : Client-side countdown için

10. Canlı Oda — Mesajlaşma

10.1 Mesajları Çekme (Polling)

Mesajları **2-3 saniyede bir** poll edin. `after` parametresi ile sadece yeni mesajları alın.

```
GET /api/room/{sessionId}/messages
GET /api/room/{sessionId}/messages?after={ISO_timestamp}
```

Başarılı Yanıt (200):

```
[
  {
    "id": "msg_1",
    "sessionId": "sess_abc123",
    "senderId": "user_abc",
    "message": "Merhaba, tarot falınıza başlayalım.",
    "createdAt": "2025-06-17T12:02:00Z"
  },
  {
    "id": "msg_2",
    "sessionId": "sess_abc123",
    "senderId": "user_456",
    "message": "Harika, hazırım!",
    "createdAt": "2025-06-17T12:02:15Z"
  }
]
```

Max 100 mesaj döner. `after` parametresi ISO timestamp formatında.

10.2 Mesaj Gönderme

POST /api/room/{sessionId}/messages

İstek Body:

```
{
  "message": "Kartlarınız çok güçlü bir enerji gösteriyor!"
}
```

Başarılı Yanıt (200):

```
{
  "id": "msg_3",
  "sessionId": "sess_abc123",
  "senderId": "user_abc",
  "message": "Kartlarınız çok güçlü bir enerji gösteriyor!",
  "createdAt": "2025-06-17T12:03:00Z"
}
```

Hata Yanıtları:

- 400 : "Message required" — Boş mesaj
- 400 : "Session is not active" — Seans aktif değil
- 403 : "Erişim reddedildi" — Kullanıcı bu seansın parçası değil

11. Canlı Oda — Timer & Ping

11.1 Timer Başlatma (Sadece Falcı)

Falcı odaya girdiğinde ve hazır olduğunda timer'ı başlatır.

PATCH /api/room/{sessionId}

İstek Body:

```
{
  "action": "start_timer"
}
```

Başarılı Yanıt (200):

```
{
  "timerStarted": true,
  "timerStartedAt": "2025-06-17T12:01:00Z"
}
```

Hata: 403 : "Only teller can start timer"

11.2 Ping (Her İki Taraf)

Her dakika gönderilir. Sunucu `minutesUsed` değerini günceller.

```
PATCH /api/room/{sessionId}
```

İstek Body:

```
{
  "action": "ping"
}
```

Başarılı Yanıt (200):

```
{
  "minutesUsed": 4,
  "timerStarted": true
}
```

Timer başlamamışsa:

```
{
  "minutesUsed": 0,
  "timerStarted": false
}
```

11.3 Client-Side Timer Mantığı

```

/// Timer'ı yönetmek için kullanılacak state:
class RoomTimerState {
    final bool timerStarted;
    final DateTime? timerStartedAt;
    final int maxMinutes;
    final int elapsedSeconds; // sunucudan alınan başlangıç değeri

    /// Kalan süreyi hesapla
    int get remainingSeconds {
        if (!timerStarted || timerStartedAt == null) return maxMinutes * 60;
        final elapsed = DateTime.now().difference(timerStartedAt!).inSeconds;
        return (maxMinutes * 60) - elapsed;
    }

    /// Yüzde olarak ilerleme
    double get progress {
        if (maxMinutes <= 0) return 0;
        final total = maxMinutes * 60;
        return (total - remainingSeconds) / total;
    }

    /// Süre doldu mu?
    bool get isExpired => remainingSeconds <= 0;

    /// Kalan süreyi "MM:SS" formatında göster
    String get formattedRemaining {
        final r = remainingSeconds.clamp(0, maxMinutes * 60);
        final m = r ~/ 60;
        final s = r % 60;
        return '${m.toString().padLeft(2, '0')}:${s.toString().padLeft(2, '0')}';
    }
}

```

Timer stratejisi:

1. Oda bilgisini al → timerStartedAt ve elapsedSeconds al
2. Client-side Timer.periodic(1 saniye) ile countdown yap
3. Her 60 saniyede bir ping gönder (sunucu minutesUsed günceller)
4. remainingSeconds <= 120 olduğunda "Süre azalıyor" uyarısı göster
5. remainingSeconds <= 0 olduğunda otomatik end çağır veya uzatma popup'ı göster

12. Canlı Oda — Süre Uzatma

12.1 Kullanıcı Tarafından Uzatma

PATCH /api/room/{sessionId}

İstek Body:

```

{
  "action": "extend",
  "minutes": 5
}

```

Arka Plan İşlemleri:

1. `minutes × creditsPerMinute` kadar jeton düşülür
2. `maxMinutes` artırılır
3. `creditsCharged` artırılır

Başarılı Yanıt (200):

```
{
  "extended": 5,
  "jetonsUsed": 50,
  "jetonsRemaining": 450,
  "newMaxMinutes": 15
}
```

Hata Yanıtları:

- 400 : "Yetersiz jeton" — Jeton yetersiz
- 403 : "Only user can extend session" — Falcı bu işlemi yapamaz

12.2 Falcı Tarafından Süre Ekleme

Falcı da kullanıcının jetonundan süre ekleyebilir.

```
PATCH /api/room/{sessionId}
```

İstek Body:

```
{
  "action": "teller_add_time",
  "minutes": 5
}
```

Başarılı Yanıt (200):

```
{
  "added": 5,
  "jetonsUsed": 50,
  "newMaxMinutes": 15,
  "userJetonsRemaining": 450
}
```

Hata Yanıtları:

- 400 : "Kullanıcının yeterli jetonu yok" — Kullanıcının jetonu yetersiz
- 403 : "Only teller can add time" — Kullanıcı bu işlemi yapamaz

13. Canlı Oda — Seans Bitirme

```
PATCH /api/room/{sessionId}
```

İstek Body:

```
{
  "action": "end"
}
```

Arka Plan İşlemleri:

1. Timer'dan gerçek kullanılan dakika hesaplanır (ceil — yukarı yuvarlama)
2. `maxMinutes` ile kısıtlanır
3. Gerçek maliyet = `actualMinutesUsed × creditsPerMinute`
4. Kullanılmayan kısım iade edilir: `creditsCharged - actualCost`
5. Komisyon hesaplanır: `actualCost × commissionRate / 100`
6. Falci kazancı: `actualCost - commission`
7. `TellerChatSession` kapatılır
8. Karşı tarafa bildirim gönderilir

Başarılı Yanıt (200):

```
{
  "ended": true,
  "actualMinutesUsed": 7,
  "actualCost": 70,
  "refundAmount": 30,
  "endedBy": "teller"
}
```

Alan	Açıklama
<code>actualMinutesUsed</code>	Gerçek kullanılan dakika (yukarı yuvarlanmış)
<code>actualCost</code>	Gerçek maliyet (jeton)
<code>refundAmount</code>	İade edilen jeton miktarı
<code>endedBy</code>	"teller" veya "user" — kim bitirdi

14. WebRTC Sinyalizasyon

WebRTC peer-to-peer bağlantı için HTTP-tabanlı sinyalizasyon sistemi kullanılır.

14.1 Sinyal Gönderme

```
POST /api/room/signal
```

İstek Body:

```
{
  "sessionId": "sess_abc123",
  "receiverId": "user_abc",
  "signalType": "offer",
  "signalData": {
    "type": "offer",
    "sdp": "v=0\r\no=- 123456789..."
  }
}
```

Alan	Tip	Zorunlu	Açıklama
sessionId	string	✓	Seans ID
receiverId	string	✓	Karşı tarafın userId'si (peerId)
signalType	string	✓	"offer", "answer", "ice-candidate"
signalData	object	✓	WebRTC sinyal verisi (SDP veya ICE candidate)

Başarılı Yanıt (200):

```
{
  "id": "sig_1",
  "sessionId": "sess_abc123",
  "senderId": "user_456",
  "receiverId": "user_abc",
  "signalType": "offer",
  "signalData": "{\"type\":\"offer\",\"sdp\":\"...\"}",
  "processed": false,
  "createdAt": "2025-06-17T12:01:05Z"
}
```

14.2 Sinyal Alma (Polling)

1-2 saniyede bir poll edin.

```
GET /api/room/signal?sessionId={sessionId}
```

Başarılı Yanıt (200):

```
[
  {
    "id": "sig_2",
    "sessionId": "sess_abc123",
    "senderId": "user_abc",
    "receiverId": "user_456",
    "signalType": "answer",
    "signalData": {
      "type": "answer",
      "sdp": "v=0\r\no=- 987654321..."
    },
    "processed": true,
    "createdAt": "2025-06-17T12:01:06Z"
  }
]
```

Sinyaller alındıktan sonra otomatik olarak `processed: true` yapılır. Bir sonraki poll'da gelmezler.

14.3 Sinyalleri Temizleme (Reconnect)

Bağlantı koptuğunda yeniden bağlanmak için eski sinyalleri temizleyin.

```
DELETE /api/room/signal?sessionId={sessionId}
```

Başarılı Yanıt (200):

```
{
  "cleared": true
}
```

14.4 WebRTC Akışı

1. Her iki taraf odaya girer
2. Bir taraf (genellikle kullanıcı) `"offer"` gönderir
 POST /api/room/signal
 { signalType: "offer", signalData: {sdp...} }
3. Karşı taraf sinyalleri poll eder
 GET /api/room/signal?sessionId=xxx
4. `"offer"` alındığında `"answer"` gönderilir
 POST /api/room/signal
 { signalType: "answer", signalData: {sdp...} }
5. ICE candidate'ler karşılıklı gönderilir
 { signalType: "ice-candidate", signalData: {...} }
6. Bağlantı kurulur → Sesli/görüntülü iletişim
7. Bağlantı koparsa:
 DELETE /api/room/signal?sessionId=xxx
 → Yeni offer/answer döngüsü başlar

15. Değerlendirme & Yorumlar

15.1 Falcının Yorumlarını Getirme

```
GET /api/fortune-tellers/{tellerId}/reviews
```

Başarılı Yanıt (200):

```
{
  "reviews": [
    {
      "id": "rev1",
      "tellerId": "clxxx1",
      "sessionId": "sess_abc123",
      "rating": 5,
      "comment": "Harika bir deneyimdi!",
      "createdAt": "2025-06-17T13:00:00Z",
      "session": {
        "user": {
          "id": "user_456",
          "name": "Mehmet K.",
          "image": null,
          "username": "mehmet42"
        },
        "fortuneType": "tarot"
      }
    }
  ],
  "averageRating": 4.8,
  "totalReviews": 180,
  "ratingDistribution": [
    { "rating": 5, "_count": { "id": 120 } },
    { "rating": 4, "_count": { "id": 40 } },
    { "rating": 3, "_count": { "id": 15 } },
    { "rating": 2, "_count": { "id": 3 } },
    { "rating": 1, "_count": { "id": 2 } }
  ]
}
```

Son 20 yorum döner, tarihe göre sıralı (en yeni önce).

16. Hediyeler & Ödüller

16.1 Falcı Ödülleri

```
GET /api/fortune-tellers/awards?tellerId={tellerId}
```

Başarılı Yanıt (200):

```
[
  {
    "id": "award1",
    "tellerId": "clxxx1",
    "awardType": "medium_of_week",
    "title": "Haftanın Medyumu",
    "awardedBy": "admin_user_id",
    "startDate": "2025-06-10T00:00:00Z",
    "endDate": "2025-06-17T00:00:00Z",
    "createdAt": "2025-06-10T..."
  }
]
```

Award Tipleri: medium_of_day, medium_of_week, medium_of_month

16.2 Falcıya Gönderilen Hediyeler

GET /api/fortune-tellers/gifts?tellerId={tellerId}

Son 7 günün hediyelerini göndericiye göre gruplanmış döner.

Başarılı Yanıt (200):

```
[
  {
    "senderId": "user_789",
    "totalAmount": 500,
    "giftCount": 3,
    "senderName": "Ali V.",
    "senderImage": "https://upload.wikimedia.org/wikipedia/commons/thumb/2/2d/Ali_Riley_during_Gotham_Angel_City_Sep_7_25-027_%28cropped%29.jpg/960px-Ali_Riley_during_Gotham_Angel_City_Sep_7_25-027_%28cropped%29.jpg",
    "senderUsername": "aliv"
  }
]
```

17. Push Notification Entegrasyonu

17.1 Falcının Alacağı Notification'lar

Olay	type	Başlık	Açıklama
Yeni talep	session_request	"Yeni Randevu Talebi"	Kullanıcı bilgisi + fal tipi + süre
Seans bitişi	session_ended	"Seans Sona Erdi"	Süre bilgisi

session_request data payload:

```
{
  "sessionId": "sess_abc123",
  "fortuneType": "tarot",
  "userName": "Mehmet K.",
  "creditsCharged": 100,
  "duration": 10
}
```

17.2 Kullanıcının Alacağı Notification'lar

Olay	type	Başlık	Açıklama
Talep kabul	session_update	"Randevu Kabul Edildi"	Odaya girin mesajı
Talep red	session_update	"Randevu Reddedildi"	Jeton iade bilgisi
Talep iptal	session_update	"Randevu İptal Edildi"	Jeton iade bilgisi
Seans bitti (falcı bitirdi)	session_ended	"Seans Sona Erdi"	Süre + iade bilgisi

session_update data payload:

```
{
  "sessionId": "sess_abc123",
  "tellerId": "clxxx1",
  "action": "accept"
}
```

17.3 Flutter'da Notification İşleme

```
/// Push notification geldiğinde:
void handleNotification(Map<String, dynamic> data) {
  final type = data['type'];
  final payload = jsonDecode(data['data'] ?? '{}');

  switch (type) {
    case 'session_request':
      // Falcı: Gelen talep popup'ı göster
      showIncomingRequestDialog(
        sessionId: payload['sessionId'],
        userName: payload['userName'],
        fortuneType: payload['fortuneType'],
        duration: payload['duration'],
      );
      break;

    case 'session_update':
      final action = payload['action'];
      if (action == 'accept') {
        // Kullanıcı: Canlı odaya yönlendir
        navigateToLiveRoom(payload['sessionId']);
      } else {
        // Kullanıcı: Red/iptal bildirimi göster
        showRefundNotification();
      }
      break;

    case 'session_ended':
      // Her iki taraf: Seans bitti ekranına yönlendir
      showSessionEndDialog();
      break;
  }
}
```

18. Tam Akış Senaryoları

Senaryo 1: Başarılı Seans (Kullanıcı Perspektifi)

```

// 1. Falcı listesini göster
final tellers = await api.get('/api/fortune-tellers?online=true');

// 2. Falcı seçildi → Seans talebi gönder
final response = await api.post('/api/fortune-tellers/session', body: {
  'tellerId': selectedTeller.id,
  'fortuneType': 'tarot',
  'duration': 10,
});
final sessionId = response['sessionId'];
// → Jeton düşüldü, status: pending
// → "Falcı onayı bekleniyor..." loading göster

// 3. Push notification ile kabul bilgisi gelir
// VEYA polling ile kontrol:
Timer.periodic(Duration(seconds: 3), (timer) async {
  final session = await api.get('/api/fortune-tellers/session?sessionId=$sessionId');
  if (session['status'] == 'active') {
    timer.cancel();
    navigateToLiveRoom(sessionId);
  } else if (session['status'] == 'cancelled') {
    timer.cancel();
    showMessage('Talep reddedildi, jetonlar iade edildi.');
```

```

    'minutes': 5,
  });

// 11. Seans bitir
final endResult = await api.patch('/api/room/${sessionId}', body: {
  'action': 'end',
});
// endResult: { ended: true, actualMinutesUsed: 7, refundAmount: 30 }

```

Senaryo 2: Başarılı Seans (Falcı Perspektifi)

```

// 1. Online ol
await api.post('/api/fortune-tellers/toggle-online', body: {'isOnline': true});

// 2. Gelen talepleri poll et
Timer.periodic(Duration(seconds: 3), (timer) async {
  final sessions = await api.get('/api/fortune-tellers/sessions?status=pending');
  if (sessions.isNotEmpty) {
    showIncomingRequestPopup(sessions.first);
  }
});

// 3. Push notification ile talep gelir → Popup göster
// Popup bilgileri: kullanıcı adı, fal tipi, süre, ödenen jeton

// 4. Kabul et
final result = await api.patch(
  '/api/fortune-tellers/sessions/${session.id}',
  body: {'action': 'accept'},
);
// result.roomId artık mevcut

// 5. Canlı odaya gir
final room = await api.get('/api/room/${session.id}');
// room.isTeller == true

// 6. Timer'ı başlat (hazır olduğunda)
await api.patch('/api/room/${session.id}', body: {'action': 'start_timer'});

// 7. Mesajlaşma + WebRTC (kullanıcıyla aynı akış)

// 8. Gerekirse süre ekle
await api.patch('/api/room/${session.id}', body: {
  'action': 'teller_add_time',
  'minutes': 5,
});

// 9. Seansı bitir
await api.patch('/api/room/${session.id}', body: {'action': 'end'});

```

Senaryo 3: Red / İptal

```

// Falcı reddeder
await api.patch(
  '/api/fortune-tellers/sessions/${session.id}',
  body: {'action': 'reject'},
);
// → Kullanıcıya jeton iadesi yapılır
// → Kullanıcıya "reddedildi" notification gider

```



19. Hata Kodları & Edge Case'ler

19.1 HTTP Hata Kodları

Kod	Endpoint	Mesaj	Çözüm
400	POST session	"Yetersiz jeton bakiyesi"	Jeton satın alma ekranına yönlendir
400	POST session	"Falcı müsait değil"	Falcı listesine dön
400	POST session	"tellerId gerekli"	Kod hatası — tellerId kontrol et
400	PATCH sessions	"Session is not pending"	Talep zaten işlenmiş
400	PATCH room (extend)	"Yetersiz jeton"	Jeton satın alma ekranına yönlendir
400	PATCH room (teller_add_time)	"Kullanıcının yeterli jetonu yok"	Kullanıcıyı bilgilendir
400	POST messages	"Session is not active"	Seans bitmiş, mesaj gönderilemez
400	POST apply	"You already have an application"	Mevcut başvuru durumunu göster
401	Tümü	"Oturum açmanız gerekiyor"	Login ekranına yönlendir
403	PATCH room (start_timer)	"Only teller can start timer"	Sadece falcı başlatabilir
403	PATCH room (extend)	"Only user can extend session"	Sadece kullanıcı uzatabilir
403	PATCH room (teller_add_time)	"Only teller can add time"	Sadece falcı ekleyebilir
403	GET/PATCH room	"Erişim reddedildi"	Kullanıcı bu seansın parçası değil
403	POST toggle-online	"Application not approved"	Başvuru onaylanmamış
403	POST toggle-online	"Account is banned"	Hesap yasaklı
404	GET room	"Session not found"	Geçersiz session ID
404	GET my-profile		Kullanıcı falcı değil

Kod	Endpoint	Mesaj	Çözüm
		"Teller profile not found"	

19.2 Edge Case'ler

Durum	Çözüm
Kullanıcı seans talebi gönderdi ama falcı yanıt vermedi	Timeout: 2-3 dakika sonra kullanıcıya iptal seçeneği sun
Her iki taraf aynı anda seans bitirdi	İlk gelen end işlenir, ikinci 400 döner (status check)
İnternet kesildi ve geri geldi	WebRTC: DELETE signals → yeni offer/answer. Mesajlar: after ile sync
Falcı timer'ı başlatmadı	Timer başlayana kadar jeton düşülmez, sadece bekleme
Süre doldu ama kimse bitirmedi	Client-side oto-end tetikle. Sunucu tarafında da ping timeout
Staff kullanıcı seans açtı	Jeton düşülmez, tüm extend/add_time da ücretsiz
Falcı başvurusu hâlâ pending	Online olamaz, seans açılmaz
Aynı falcıya aynı anda 2 kullanıcı talep gönderdi	Her ikisi de pending olur, falcı sırayla kabul/ret yapar

20. Flutter Kod İskeletleri

20.1 API Service

```

class FortuneTellerService {
    final ApiClient _api; // Mevcut JWT-tabanlı HTTP client

    // === Falcı Listeleme ===
    Future<List<LiveFortuneTeller>> getTellers({
        bool? onlineOnly,
        String? specialty,
        String? sort,
    }) async {
        final params = <String, String>{};
        if (onlineOnly == true) params['online'] = 'true';
        if (specialty != null) params['specialty'] = specialty;
        if (sort != null) params['sort'] = sort;

        final response = await _api.get('/api/fortune-tellers', queryParams: params);
        return (response as List).map((e) => LiveFortuneTeller.fromJson(e)).toList();
    }

    Future<LiveFortuneTeller> getTellerProfile(String tellerId) async {
        final response = await _api.get('/api/fortune-tellers/$tellerId');
        return LiveFortuneTeller.fromJson(response);
    }

    // === Başvuru ===
    Future<Map<String, dynamic>> applyAsTeller({
        required String displayName,
        required List<String> specialties,
        String? bio,
        String? applicationNote,
    }) async {
        return await _api.post('/api/fortune-tellers/apply', body: {
            'displayName': displayName,
            'specialties': specialties,
            if (bio != null) 'bio': bio,
            if (applicationNote != null) 'applicationNote': applicationNote,
        });
    }

    // === Falcı Profil ===
    Future<Map<String, dynamic>> getMyProfile() async {
        return await _api.get('/api/fortune-tellers/my-profile');
    }

    Future<Map<String, dynamic>> toggleOnline({bool? isOnline}) async {
        return await _api.post('/api/fortune-tellers/toggle-online',
            body: isOnline != null ? {'isOnline': isOnline} : {});
    }

    Future<Map<String, dynamic>> getOnlineStatus() async {
        return await _api.get('/api/fortune-tellers/toggle-online');
    }

    // === Seans Oluşturma ===
    Future<Map<String, dynamic>> createSession({
        required String tellerId,
        String fortuneType = 'general',
        int duration = 10,
    }) async {
        return await _api.post('/api/fortune-tellers/session', body: {
            'tellerId': tellerId,
            'fortuneType': fortuneType,
            'duration': duration,
        });
    }
}

```

```

    });
}

Future<Map<String, dynamic>> getSessionDetails(String sessionId) async {
    return await _api.get('/api/fortune-tellers/session?sessionId=$sessionId');
}

Future<List<Map<String, dynamic>>> getMySessions() async {
    final response = await _api.get('/api/fortune-tellers/session');
    return List<Map<String, dynamic>>.from(response);
}

// === Falcı: Gelen Talepler ===
Future<List<Map<String, dynamic>>> getPendingSessions() async {
    final response = await _api.get('/api/fortune-tellers/sessions?status=pending');
    return List<Map<String, dynamic>>.from(response);
}

Future<Map<String, dynamic>> respondToSession(String sessionId, String action)
async {
    return await _api.patch('/api/fortune-tellers/sessions/$sessionId', body: {
        'action': action, // 'accept', 'reject', 'cancel'
    });
}

// === Aktif Seanslar ===
Future<List<Map<String, dynamic>>> getActiveSessions() async {
    final response = await _api.get('/api/user/active-sessions');
    return List<Map<String, dynamic>>.from(response);
}

// === Oda ===
Future<Map<String, dynamic>> getRoomInfo(String sessionId) async {
    return await _api.get('/api/room/$sessionId');
}

Future<Map<String, dynamic>> roomAction(String sessionId, String action, {int? minutes})
async {
    final body = <String, dynamic>{'action': action};
    if (minutes != null) body['minutes'] = minutes;
    return await _api.patch('/api/room/$sessionId', body: body);
}

// === Mesajlar ===
Future<List<Map<String, dynamic>>> getMessages(String sessionId, {String? after})
async {
    final url = '/api/room/$sessionId/messages' + (after != null ? '?after=$after' : '');
    final response = await _api.get(url);
    return List<Map<String, dynamic>>.from(response);
}

Future<Map<String, dynamic>> sendMessage(String sessionId, String message) async {
    return await _api.post('/api/room/$sessionId/messages', body: {
        'message': message,
    });
}

// === WebRTC Sinyal ===
Future<void> sendSignal({
    required String sessionId,
    required String receiverId,
    required String signalType,

```

```

        required Map<String, dynamic> signalData,
    }) async {
        await _api.post('/api/room/signal', body: {
            'sessionId': sessionId,
            'receiverId': receiverId,
            'signalType': signalType,
            'signalData': signalData,
        });
    }

    Future<List<Map<String, dynamic>>> pollSignals(String sessionId) async {
        final response = await _api.get('/api/room/signal?sessionId=$sessionId');
        return List<Map<String, dynamic>>.from(response);
    }

    Future<void> clearSignals(String sessionId) async {
        await _api.delete('/api/room/signal?sessionId=$sessionId');
    }

    // === Yorumlar ===
    Future<Map<String, dynamic>> getTellerReviews(String tellerId) async {
        return await _api.get('/api/fortune-tellers/$tellerId/reviews');
    }

    // === Hediyeler & Ödüller ===
    Future<List<Map<String, dynamic>>> getTellerAwards(String tellerId) async {
        final response = await _api.get('/api/fortune-tellers/awards?tellerId=$tellerId');
        return List<Map<String, dynamic>>.from(response);
    }

    Future<List<Map<String, dynamic>>> getTellerGifts(String tellerId) async {
        final response = await _api.get('/api/fortune-tellers/gifts?tellerId=$tellerId');
        return List<Map<String, dynamic>>.from(response);
    }
}

```

20.2 Canlı Oda Controller

```

class LiveRoomController extends ChangeNotifier {
  final FortuneTellerService _service;
  final String sessionId;

  Map<String, dynamic>? roomInfo;
  List<Map<String, dynamic>> messages = [];
  bool isLoading = true;
  bool isTimerStarted = false;
  DateTime? timerStartedAt;
  int maxMinutes = 0;
  int elapsedSeconds = 0;
  bool isUser = false;
  bool isTeller = false;
  String? peerId;

  Timer? _pingTimer;
  Timer? _messageTimer;
  Timer? _signalTimer;
  Timer? _countdownTimer;
  String? _lastMessageTime;

  LiveRoomController(this._service, this.sessionId);

  Future<void> init() async {
    isLoading = true;
    notifyListeners();

    try {
      roomInfo = await _service.getRoomInfo(sessionId);
      isUser = roomInfo!['isUser'] ?? false;
      isTeller = roomInfo!['isTeller'] ?? false;
      peerId = roomInfo!['peerId'];
      isTimerStarted = roomInfo!['timerStarted'] ?? false;
      timerStartedAt = roomInfo!['timerStartedAt'] != null
        ? DateTime.parse(roomInfo!['timerStartedAt'])
        : null;
      maxMinutes = roomInfo!['maxMinutes'] ?? 0;
      elapsedSeconds = roomInfo!['elapsedSeconds'] ?? 0;

      // İlk mesajları çek
      messages = await _service.getMessages(sessionId);
      if (messages.isNotEmpty) {
        _lastMessageTime = messages.last['createdAt'];
      }

      // Polling başlat
      _startPolling();

      // Timer başlamışsa countdown başlat
      if (isTimerStarted) {
        _startCountdown();
      }
    } catch (e) {
      print('Room init error: $e');
    }

    isLoading = false;
    notifyListeners();
  }

  void _startPolling() {
    // Mesaj polling (her 2 saniye)
  }
}

```

```

    _messageTimer = Timer.periodic(Duration(seconds: 2), (_) => _pollMessages());

    // Ping (her 60 saniye)
    _pingTimer = Timer.periodic(Duration(seconds: 60), (_) => _sendPing());

    // Sinyal polling (her 1.5 saniye)
    _signalTimer = Timer.periodic(Duration(milliseconds: 1500), (_) => _pollSignals());
  }

  void _startCountdown() {
    _countdownTimer?.cancel();
    _countdownTimer = Timer.periodic(Duration(seconds: 1), (_) {
      notifyListeners(); // UI'ı her saniye güncelle
    });
  }

  int get remainingSeconds {
    if (!isTimerStarted || timerStartedAt == null) return maxMinutes * 60;
    final elapsed = DateTime.now().difference(timerStartedAt!).inSeconds;
    return (maxMinutes * 60) - elapsed;
  }

  String get formattedTime {
    final r = remainingSeconds.clamp(0, maxMinutes * 60);
    return '${(r ~/ 60).toString().padLeft(2, '0')}:${(r % 60).toString().padLeft(2, '0')}'
  }

  bool get isTimeRunningLow => remainingSeconds <= 120 && remainingSeconds > 0;
  bool get isTimeExpired => isTimerStarted && remainingSeconds <= 0;

  // === Falcı: Timer Başlat ===
  Future<void> startTimer() async {
    final result = await _service.roomAction(sessionId, 'start_timer');
    isTimerStarted = true;
    timerStartedAt = DateTime.parse(result['timerStartedAt']);
    _startCountdown();
    notifyListeners();
  }

  // === Kullanıcı: Süre Uzat ===
  Future<Map<String, dynamic>> extendSession(int minutes) async {
    final result = await _service.roomAction(sessionId, 'extend', minutes: minutes);
    maxMinutes = result['newMaxMinutes'];
    notifyListeners();
    return result;
  }

  // === Falcı: Süre Ekle ===
  Future<Map<String, dynamic>> tellerAddTime(int minutes) async {
    final result = await _service.roomAction(sessionId, 'teller_add_time', minutes: minutes);
    maxMinutes = result['newMaxMinutes'];
    notifyListeners();
    return result;
  }

  // === Seans Bitir ===
  Future<Map<String, dynamic>> endSession() async {
    final result = await _service.roomAction(sessionId, 'end');
    _dispose();
    return result;
  }

```

```

}

// === Mesaj Gönder ===
Future<void> sendMessage(String text) async {
  final msg = await _service.sendMessage(sessionId, text);
  messages.add(msg);
  _lastMessageTime = msg['createdAt'];
  notifyListeners();
}

Future<void> _pollMessages() async {
  try {
    final newMsgs = await _service.getMessages(sessionId, after: _lastMessageTime);
    if (newMsgs.isNotEmpty) {
      messages.addAll(newMsgs);
      _lastMessageTime = newMsgs.last['createdAt'];
      notifyListeners();
    }
  } catch (_) {}
}

Future<void> _sendPing() async {
  try {
    final result = await _service.roomAction(sessionId, 'ping');
    // Timer falcı tarafından başlatıldıysa güncelle
    if (result['timerStarted'] == true && !isTimerStarted) {
      isTimerStarted = true;
      // Room bilgisini yenile timerStartedAt almak için
      final freshRoom = await _service.getRoomInfo(sessionId);
      timerStartedAt = freshRoom['timerStartedAt'] != null
        ? DateTime.parse(freshRoom['timerStartedAt'])
        : null;
      maxMinutes = freshRoom['maxMinutes'] ?? maxMinutes;
      _startCountdown();
      notifyListeners();
    }
  } catch (_) {}
}

Future<void> _pollSignals() async {
  try {
    final signals = await _service.pollSignals(sessionId);
    for (final signal in signals) {
      // WebRTC sinyali işle
      onSignalReceived?.call(signal);
    }
  } catch (_) {}
}

// WebRTC sinyal callback
Function(Map<String, dynamic>)? onSignalReceived;

void _dispose() {
  _pingTimer?.cancel();
  _messageTimer?.cancel();
  _signalTimer?.cancel();
  _countdownTimer?.cancel();
}

@override
void dispose() {
  _dispose();
  super.dispose();
}

```

```
}  
}
```

20.3 WebRTC Manager

```

class WebRTCManager {
    final FortuneTellerService _service;
    final String sessionId;
    final String peerId;

    RTCPeerConnection? _peerConnection;
    MediaStream? _localStream;
    MediaStream? _remoteStream;

    Function(MediaStream)? onRemoteStream;
    Function(MediaStream)? onLocalStream;

    WebRTCManager(this._service, this.sessionId, this.peerId);

    Future<void> init({bool video = true, bool audio = true}) async {
        // 1. Local stream al
        _localStream = await navigator.mediaDevices.getUserMedia({
            'video': video,
            'audio': audio,
        });
        onLocalStream?.call(_localStream!);

        // 2. Peer connection oluştur
        _peerConnection = await createPeerConnection({
            'iceServers': [
                {'urls': 'stun:stun.l.google.com:19302'},
                {'urls': 'stun:stun1.l.google.com:19302'},
            ]
        });

        // Local track'leri ekle
        _localStream!.getTracks().forEach((track) {
            _peerConnection!.addTrack(track, _localStream!);
        });

        // Remote stream dinle
        _peerConnection!.onTrack = (event) {
            if (event.streams.isNotEmpty) {
                _remoteStream = event.streams[0];
                onRemoteStream?.call(_remoteStream!);
            }
        };

        // ICE candidate gönder
        _peerConnection!.onIceCandidate = (candidate) {
            if (candidate.candidate != null) {
                _service.sendSignal(
                    sessionId: sessionId,
                    receiverId: peerId,
                    signalType: 'ice-candidate',
                    signalData: candidate.toMap(),
                );
            }
        };

        /// Arama başlat (offer gönder)
        Future<void> createOffer() async {
            final offer = await _peerConnection!.createOffer();
            await _peerConnection!.setLocalDescription(offer);

            await _service.sendSignal(

```

```

        sessionId: sessionId,
        receiverId: peerId,
        signalType: 'offer',
        signalData: offer.toMap(),
    );
}

/// Gelen sinyali işle
Future<void> handleSignal(Map<String, dynamic> signal) async {
    final type = signal['signalType'];
    final data = signal['signalData'];

    switch (type) {
        case 'offer':
            await _peerConnection!.setRemoteDescription(
                RTCSessionDescription(data['sdp'], data['type']),
            );
            final answer = await _peerConnection!.createAnswer();
            await _peerConnection!.setLocalDescription(answer);
            await _service.sendSignal(
                sessionId: sessionId,
                receiverId: peerId,
                signalType: 'answer',
                signalData: answer.toMap(),
            );
            break;

        case 'answer':
            await _peerConnection!.setRemoteDescription(
                RTCSessionDescription(data['sdp'], data['type']),
            );
            break;

        case 'ice-candidate':
            await _peerConnection!.addCandidate(
                RTCIceCandidate(
                    data['candidate'],
                    data['sdpMid'],
                    data['sdpMLineIndex'],
                ),
            );
            break;
    }
}

/// Yeniden bağlan
Future<void> reconnect() async {
    await _service.clearSignals(sessionId);
    await _peerConnection?.close();
    await init();
    await createOffer();
}

void dispose() {
    _localStream?.dispose();
    _remoteStream?.dispose();
    _peerConnection?.close();
}
}

```

20.4 Falcı Gelen Talep Widget'ı

```

class IncomingRequestDialog extends StatelessWidget {
  final Map<String, dynamic> session;
  final FortuneTellerService service;

  Future<void> _accept(BuildContext context) async {
    try {
      await service.respondToSession(session['id'], 'accept');
      Navigator.of(context).pop();
      // Canlı odaya yönlendir
      Navigator.of(context).push(
        MaterialPageRoute(
          builder: (_) => LiveRoomScreen(sessionId: session['id']),
        ),
      );
    } catch (e) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Hata: $e')),
      );
    }
  }

  Future<void> _reject(BuildContext context) async {
    try {
      await service.respondToSession(session['id'], 'reject');
      Navigator.of(context).pop();
    } catch (e) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Hata: $e')),
      );
    }
  }
}

@override
Widget build(BuildContext context) {
  final fortuneTypes = {
    'coffee': 'Kahve Falı', 'tarot': 'Tarot', 'astrology': 'Astroloji',
    'palmistry': 'El Falı', 'numerology': 'Numeroloji', 'general': 'Genel',
  };

  return AlertDialog(
    title: Text('🌐 Yeni Randevu Talebi'),
    content: Column(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        // Kullanıcı bilgisi
        CircleAvatar(
          backgroundImage: session['user']['image'] != null
            ? NetworkImage(session['user']['image']) : null,
          child: session['user']['image'] == null
            ? Text(session['user']['name'][0] ?? '?') : null,
        ),
        SizedBox(height: 8),
        Text(session['user']['name'] ?? 'Anonim', style: TextStyle(fontWeight: FontWeight.bold)),
        SizedBox(height: 16),
        // Detaylar
        Text('Fal Tipi: ${fortuneTypes[session['fortuneType']] ?? session['fortune-
Type']}'),
        Text('Süre: ${session['maxMinutes']} dakika'),
        Text('Ödeme: ${session['creditsCharged']} jeton'),
      ],
    ),
  );
}

```

```

actions: [
  TextButton(
    onPressed: () => _reject(context),
    child: Text('Reddet', style: TextStyle(color: Colors.red)),
  ),
  ElevatedButton(
    onPressed: () => _accept(context),
    child: Text('Kabul Et'),
  ),
],
);
}
}

```

ÖNEMLİ NOTLAR

Auth Header

Tüm API çağrılarında:

```
Authorization: Bearer {JWT_TOKEN}
```

Base URL

```
https://canlifal.com
```

Polling Aralıkları

Ne	Aralık	Kim
Bekleyen talepler	3-5 sn	Falci (çevrimiçi)
Mesajlar	2-3 sn	Her iki taraf (odadayken)
WebRTC sinyalleri	1-1.5 sn	Her iki taraf (odadayken)
Ping	60 sn	Her iki taraf (odadayken)
Seans durumu	3 sn	Kullanıcı (bekleme ekranında)

Jeton Hesaplama Formülü

```

toplam_maliyet = süre_dakika ☐ jeton_dakika_başına (varsayılan: 10)
komisyon = toplam_maliyet ☐ komisyon_oranı / 100 (varsayılan: %20)
falci_kazancı = toplam_maliyet - komisyon
iade = toplam_ödenen - gerçek_kullanılan_maliyet

```

Durum Geçişleri

```
pending → active      (falcı kabul etti)
pending → cancelled   (falcı reddetti/iptal etti)
active  → completed   (seans bitti – end action)
active  → cancelled   (cancel action)
```

Bu prompt canlifal.com backend API'leri ile tam uyumludur.

Tüm endpoint'ler, request/response formatları ve iş mantığı gerçek backend kodundan çıkarılmıştır.